Sinusoidal Feature Embeddings for Neural Renormalization Group Flows on the Ising Models

Ao Wang
High School Student
Shanghai Foreign Language School Affiliated to SISU
Shanghai, China
aowang111@gmail.com

Peter Lu*
Electrical and Computer Engineering
Tufts University
Medford, United States
peter.lu@tufts.edu

Oreoluwa Alao
Computer Science and Electrical Engineering
Massachusetts Institute of Technology
Cambridge, United States
orealao@mit.edu

***Abstract*—Continuous normalizing flows are a powerful tool for generative models, but their computational cost is dominated by adaptive solver function evaluations (NFE). We investigate two strategies for reducing this cost, which includes constructing the flow from sinusoidal components, and penalizing long and curved transformation paths to optimize transport and learn smooth paths. Then we evaluate their performance on the 2D checkerboard benchmark, the sinusoidal embedding matches the tanh baseline in sample quality and likelihood while reducing NFE by ~25%; adding a kinetic-energy regularizer further lowers ordinary differential equation solver cost. Finally, we extend neural renormalization group (RG) for the Ising models, the two activations perform equally in 1D, but the sinusoidal embedding yields significantly lower negative log-likelihood at fixed transport cost in 2D.**

***Keywords*—sinusoidal embedding, normalizing flows, optimal transport paths, kinetic-energy regularizer, Ising model**

I. INTRODUCTION

Generative models avoid the high cost of decorrelating traditional samplers by learning the data distribution and generating independent samples from noise. Normalizing flows do this via invertible mappings from a simple prior to the target density [1]. Classical flows keep the Jacobian determinant of this map tractable by constraining their architectures [2]. Continuous normalizing flows (CNFs) replace restricted flows with a neural ordinary differential equation (ODE) [3]. FFJORD [4] introduced the first fully unrestricted CNF but suffered from unbounded the number of function evaluations (NFE) growth and noisy trace estimation.

The renormalization group (RG) is an exact flow in theory space. Its optimal coarse-graining and its multiscale perspective exposes sharp localized structure in physical data across all scales [5,6]. RNODE added a kinetic-energy penalty to straighten trajectories, yet still relied on stochastic trace estimators and incurred per-step regularization costs [7]. OT-Flow replaced the stochastic trace with an exact, noise-free computation, but its activation function was not tailored to learning the scalar potential [8]. These advances improve CNF efficiency via loss or solver tuning, but largely overlook the velocity-field architecture.

Sinusoidal input embedding at tunable frequency ω_0 lets a standard network represent sharp signals far better than an equal-sized tanh network [9,10]. A sinusoidal velocity field could help a CNF by fitting sharp densities with gentler dynamics, or hurt it by injecting high-frequency stiffness. Which effect dominates has not been studied systematically. We investigate this question on a sharp 2D benchmark and on the Ising models, comparing sinusoidal-embedding velocity fields against tanh baselines under controlled conditions..

II. BACKGROUND AND PRELIMINARIES

In this section, we introduce the core concepts that appear throughout the paper.

A. Normalizing Flows and Their Solver Cost

In a normalizing flow, the exact probability density that the model assigns to any observed data point x [4]:

$$\log p_x(x) = \log p_z\left(f^{-1}(x)\right) + \log\left|\det \frac{\partial f^{-1}(x)}{\partial x}\right| \tag{1}$$

where $\partial f^{-1}(x)/\partial x$ is the Jacobian determinant of f in d dimensions costs $o\left(d^3\right)$, which is proportional to the cube of dimension. Discrete flows keep this determinant cheap by restricting their layers so the Jacobian is triangular, as in RealNVP [2].

B. Sinusoidal Networks and Optimal Transport

A sinusoidal-feature network encodes the input through a sinusoidal embedding $x \mapsto \sin\left(\omega_0\left(W_0 x + b_0\right)\right)$ at a frequency scale ω_0, which spreads the input across many periods and allows the subsequent network to represent sharp, high-frequency structure [9,10]. Its dynamic (Benamou–Brenier) form minimizes the kinetic energy of the flow, yielding straight, constant-speed trajectories [11].

$$\begin{cases} W_2^2\left(p_0, p_1\right) = \min\limits_{v} \int_0^1 \int \frac{1}{2}\left\|v(z,t)\right\|^2 p_t(z)\, dz dt \\ \text{s.t.} \quad \partial_t p_t + \nabla \cdot \left(p_t v\right) = 0 \end{cases} \tag{2}$$

C. Neural Renormalization Group

For the Ising model, renormalization group coarse-grains neighboring spins into blocks to reveal long-length-scale behavior. The structure of the NeuralRG network as described in [12], compressing a configuration x into latent variables $z = g^{-1}(x)$, or generation to reconstruct a configuration $x = g(z)$ from Gaussian noise. Each layer is a bijection built from small local flows, including a disentangler that de-correlates neighboring sites, followed by a decimator that separates one coarse variable from the rest. The composition is exactly invertible and its likelihood is exact.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

D. Notation

For the flow experiments, we model data on $\mathbb{R}^2$. Let $z(t)$ be the state at artificial time $t \in [0,T]$, with $z(0) \sim \mathrm{N}(0, I_2)$ where I_2 is the 2×2 identity, and let $v_\theta(z,t)$ be the velocity field with parameters θ. The flow and its log-density evolve as following the instantaneous change-of-variables formula [3]:

$$\frac{dz(t)}{dt} = v_\theta(z(t),t), \quad \frac{d\log p(z(t))}{dt} = -tr\left(\frac{\partial v_\theta}{\partial z}\right) \tag{3}$$

Integrating Eq. (3) backward from $z(T) = x$ gives $\log p_\theta(x)$, and we write $L_{NLL} = -\mathrm{E}_x[\log p_\theta(x)]$. We denote by ω0 the sinusoidal embedding frequency scale and by $\alpha \geq 0$ the weight of the kinetic-energy penalty, as Eq. (2), with L_{NLL} as the objective to be minimized in our research.

III. Methodology

In this section, we build a continuous normalizing flow and train it by minimizing the negative log-likelihood. We design two velocity-field architectures, a tanh baseline and a sinusoidal embedding alternative, with all other design choices held fixed. We also add a kinetic-energy penalty to further reduce solver cost. Finally, we embed the flow into a neural renormalization group to evaluate it on the Ising model.

A. Continuous Normalizing Flow

We integrate the augmented ODE Eq. (3) with state (z, log p) from t = T to $t = 0$ to evaluate the exact likelihood of a data point, and from $t = 0$ to $t = T$ to generate a sample from Gaussian noise. In two dimensions the trace $tr(\partial v_\theta/\partial z)$ is obtained exactly from two vector-Jacobian products, one per coordinate, avoiding the variance of the stochastic estimator used in higher dimensions [4]. We train by minimizing L_{NLL}.

B. Velocity-field Architectures

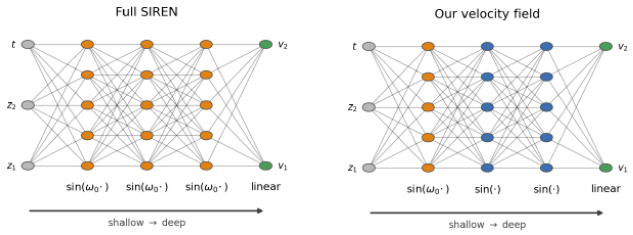
We compare two velocity fields of matched width and depth, so that any difference in behavior is attributable to the activation alone. Both map the concatenated input (z, t) to a velocity in $\mathbb{R}^2$. The baseline follows FFJORD [4], stacking concatenation–squash layers as:

$$CS(\tau, y) = (W_1 y) \odot \sigma(W_2 \tau) + W_3 \tau \tag{4}$$

with tanh nonlinearities between layers and a linear output, so time τ modulates every layer through a learned gain and bias, as shown in Eq. (4). The alternative is a sinusoidal-feature network, in which the frequency scale ω0 acts only at the input embedding:

$$h_1 = \sin\left(\omega_0\left(W_0(z,t) + b_0\right)\right), \quad h_{\ell+1} = \sin\left(W_\ell h_\ell + b_\ell\right) \tag{5}$$

with a linear output layer and the sinusoidal initialization of [10]: first-layer weights from $U(-1/n, 1/n)$ and later weights from $U\left(-\sqrt{6/n}, \sqrt{6/n}\right)$, where n is fan-in. The learnable phase bias b_0 lets a sine-only map span both sine and cosine components (since $\cos\theta = \sin(\theta + \pi/2)$). We treat ω_0 as primary hyperparameter.



(a) (b)

Fig. 1. Velocity-field architectures: (a) A full SIREN [10]. (b) Our proposed velocity field.

Fig. 1(a) contrasts a full SIREN which applies a sinusoidal activation at every layer (orange). We proposed velocity field with unit-frequency sinusoidal layers $\sin(\cdot)$ thereafter (blue) and a linear output (green), as shown in Fig. 1(b). We apply the tunable frequency ω_0 only at the input embedding and unit-frequency sinusoidal layers thereafter, as defined in Eq. (5). Both map the concatenated input (z, t) to a velocity.

C. Kinetic-energy Regularization

To decouple the sinusoidal embedding's representational benefit from its stiffness cost, we add the Benamou–Brenier kinetic energy of the flow to the objective [11],

$$L = L_{NLL} + \alpha \mathrm{E}\left[\int_0^T \frac{1}{2}\left\|v_\theta(z(t),t)\right\|^2 dt\right] \tag{6}$$

which, among fields that fit the data, favors the one of least kinetic energy. We estimate the integral in Eq. (6) by a stratified Monte Carlo average over m time points drawn one per equal sub-interval of [0, T], reading the states $z(t_j)$ off the same solve used for the likelihood, so the penalty adds no function evaluations.

D. A Neural RG with Continuous-flow Blocks for Ising Modes

We instantiate the neural RG with the efficient flow above as its per-block velocity field. A layer first applies a one-site cyclic offset, then a disentangler CNF on each block, then a decimator CNF that maps each block to one coarse variable and c decimated variables (c=1 in 1D, c=3 in 2D). Writing B_ℓ for the layer:

$$(x_{coarse}, \xi) = \mathrm{B}_\ell(x), \quad \xi \sim \mathrm{N}(0, I) \tag{7}$$

where x_{coarse} is passed to the next, coarser layer and the decimated ξ are regularized toward a standard normal. Stacking $\log_2 L$ such layers maps an L- (or $L \times L$-) site configuration to a fully Gaussian latent code, and running the layers in reverse generates a configuration from noise.

Fig. 2 illustrates the architecture. Light-gray blocks denote disentanglers, which reduce correlations among inputs; dark-gray blocks denote decimators, which pass only a subset of outputs to the next layer while treating the remainder as latent variables [12]. The Benamou–Brenier kinetic-energy penalty is applied block-wise during training. Because the Boltzmann distribution of the Ising model is known in closed form, such a model can also be trained variationally by minimizing the free energy, with no dataset required. Here we train by maximum likelihood on Monte Carlo samples, so that the 1D and 2D comparisons parallel the checkerboard study.

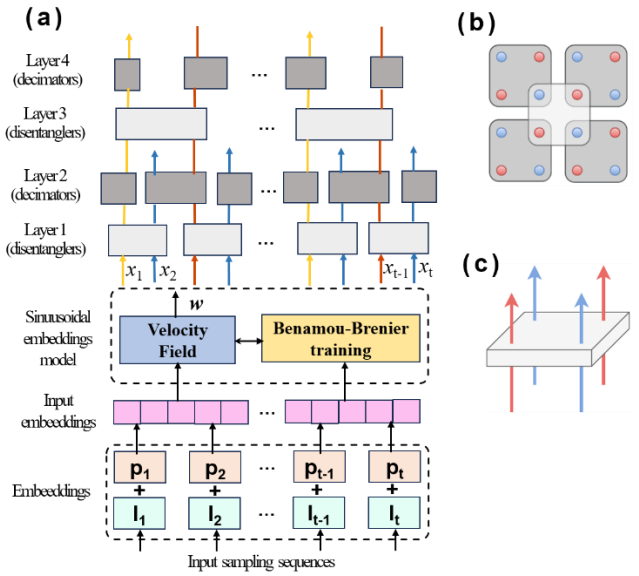


Fig. 2. (a) Proposed scheme of sinusoidal feature embeddings for neural renormalization group flows. (b) Arrangement of bijectors [12]. (c) Each bijector acts on four variables [12].

E. Experimental Setup

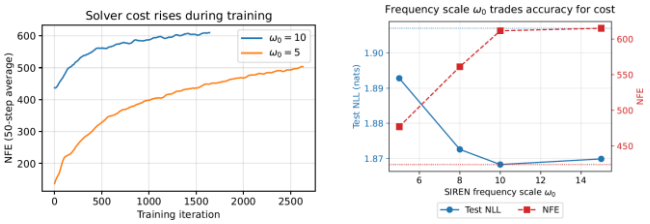
We evaluate our flow on two tasks: checkerboard density (sharp-boundary benchmark) and critical Ising models (1D and 2D). Metrics include NLL, squared MMD, and NFE for flows; kinetic energy is additionally reported for RG. Training uses Adam with early stopping and budget-aware or Bayesian hyperparameter optimization. Flow study uses 8-tile checkerboard density (hard gaps, sharp gradients) with 10,000 training, 2,000 validation, and 2,000 test points, standardized to training-set statistics. RG study uses critical 1D (L=32) and 2D (16×16) Ising models, sampled via Metropolis Monte Carlo and mapped to continuous variables by the standard relaxatio.

Flow models are trained with Adam [13] at learning rate 10^{-3} and batch size 512 for up to 10,000 iterations. The hyperparameters ω_0, the kinetic weight α, the time treatment, width, and depth are chosen by a budget-aware search, and every final comparison is repeated over three random seeds. The RG hyperparameters (penalty coefficient, marginal-regularization weight, and its decay schedule) are tuned by Bayesian optimization. We implement all models in JAX [14] using the Equinox and Diffrax libraries [15], Integrating Eq. (3) with the adaptive Tsit5 solver at relative and absolute tolerances 10^{-5} and a step-count cap of 1000.

IV. Results and Discussion

A. The Frequency Scale Governs Stiffness

Stiffness is controlled by sinusoidal-embedding frequency scale ω_0, as shown in Fig. 3. At the image-fitting default ω_0 = 30 the flow is severely over-stiff: training is unstable, the cost exceeds 2000 NFE, and generated samples. Reducing ω_0 into the range 5–10 removes this pathology entirely. Within this range ω_0 traces a smooth accuracy–cost frontier. Fig. 3(a) shows the solver cost during training for two representative settings: the stiffer field (ω_0=10) grows costlier to integrate, while the tamer field (ω_0=5) stays cheaper. Fig. 3(b) quantifies the trade-off across $\omega_0 \in \{5, 8, 10, 15\}$: increasing ω_0 improves NLL from 1.8928 to 1.8683 while raising NFE from 477 to 615. Every setting we tried already improves on the tanh baseline NLL of 1.907, and all reach the MMD floor.



(a) (b)

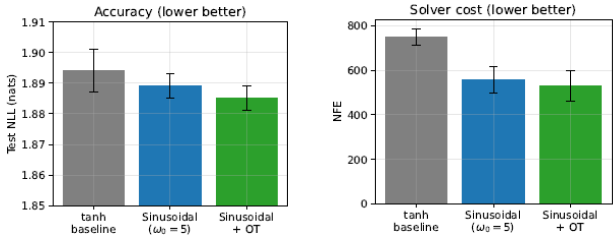
Fig. 3. (a) Solver cost (NFE) during training for two sinusoidal-embedding flows. (b) The frequency scale ω_0 trades accuracy against cost.

B. Sinusoidal Embedding Matches Baseline at Lower Cost

We compare the tuned sinusoidal-embedding flow (ω_0=5) against the tanh baseline under the three-seed protocol, as shown in Table I and Fig. 4. The unbiased estimator of squared MMD may be negative [16]. Their NLL and MMD differ by less than one standard deviation, and both reach the MMD floor. On solver cost the sinusoidal-embedding flow is significantly cheaper. Its NFE (557±58) lies below the baseline's (748±38) with non-overlapping error bars, a 25% reduction. This is not bought with extra capacity. The flow uses 14% fewer parameters and reaches its best performance in about a third as many iterations. Fig. 5 confirms that the lower cost reproduces all eight tiles with clean boundaries and no outlying points.

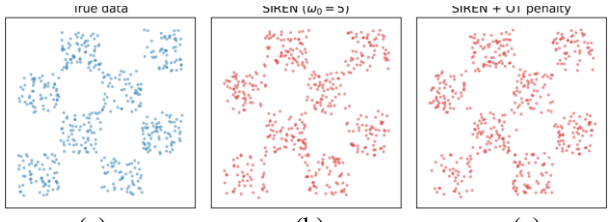
TABLE I. Main Comparison on the Checkerboard Density

Model	Test NLL	NFE	$MMD^2(\times 10^{-4})$	Params
tanh baseline	1.894±0.007	748±38	−3.1±2.4	40,228
Sinusoidal (ω_0=5)	1.889±0.004	557±58	−2.3±5.0	34,690
Sinusoidal+OT (α=0.1)	1.889±0.006	531±54	−0.3±3.1	34,690



(a) (b)

Fig. 4. (a) Three-seed comparison. (b) solver cost is significantly lower for the sinusoidal-embedding flows. Error bars are one standard deviation.



(a) (b) (c)

Fig. 5. Generated samples: (a) true data (b) the tuned sinusoidal embedding flow (c) the same flow with the transport penalty.

C. The Optimal-transport Penalty

Adding the kinetic-energy penalty at a moderate weight reduces the solver cost at essentially unchanged likelihood under single seed, as shown in Fig. 6. At the best single-seed setting α=0.1 the NFE falls from 476 to 439 at unchanged NLL. Over three seeds (Table I) this attains 531±54 NFE, below the unregularized flow's 557±58. The dominant efficiency gain therefore comes from the sinusoidal-embedding architecture, with the transport penalty a smaller, consistent improvement.

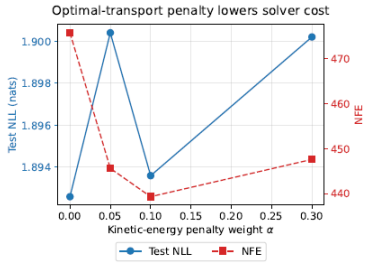
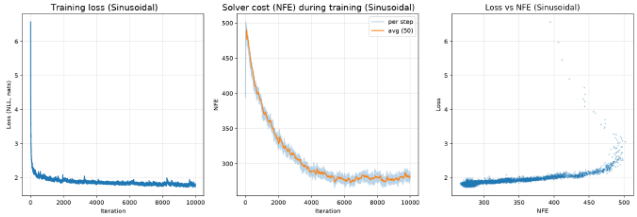


Fig. 6. Effect of the kinetic-energy penalty weight α.

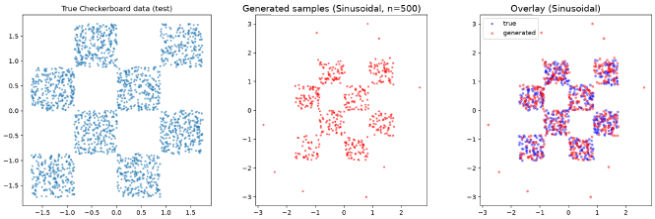
Fig. 7 collects the full training and generation diagnostics for sinusoidal-embedding flow with the kinetic-energy penalty with 10,000 training iterations. Fig. 7(a) shows the training loss decreases smoothly to a plateau near 1.8 nats. Fig. 7(b) indicates the solver cost decreases over training. The NFE falls from roughly 500 to about 280 at convergence, as the penalty straightens the flow. Fig. 7(c) shows the NLL–NFE trajectory settles into the low-cost, low-NLL corner.



(a) (b) (c)

Fig. 7. Full diagnostics for the sinusoidal-embedding flow with the kinetic-energy penalty: (a) training loss, (b) solver cost (NFE) over training, (c) the NLL–NFE trajectory.

Fig. 8(a) shows the true checkerboard test data. Fig. 8(b) displays 500 samples generated by the trained flow and indicates that all eight modes are reproduced without dropping. Fig. 8(c) overlays the generated samples (red) on the true data (blue), revealing close coincidence across all modes. Together with Fig. 7, the kinetic-energy penalty reduces solver cost while preserving both the likelihood and the visual sample quality.



(a) (b) (c)

Fig. 8. Generation fidelity for the sinusoidal-embedding flow with the kinetic-energy penalty: (a) true checkerboard data, (b) generated samples, (c) overlay of true and generated samples.

D. Ablation: Representing Time

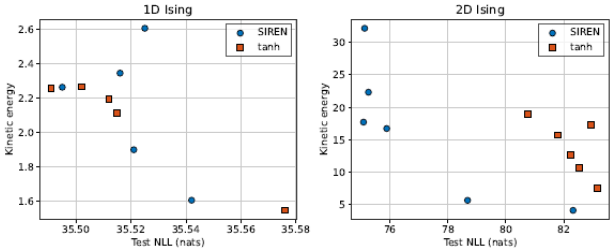
We justify treatment time by ablating the three alternatives at fixed ω_0 under single seed as shown in Table II. A plain normalized scalar and a Fourier embedding both train stably and produce clean samples, with the embedding slightly better in likelihood. Per-layer modulation is worse on every axis. We adopt the Fourier embedding throughout.

TABLE II. ABLATION OF THE TIME REPRESENTATION (Ω0=10).

Time treatment	Test NLL	NFE	MMD²(×10⁻⁴)
Scalar	1.8812	600	−0.9
Fourier embedding	1.8720	610	6.1
Per-layer modulation	1.9033	798	0.8

E. The Neural RG on the Ising Models

Comparing sinusoidal embedding and tanh blocks on the 1D and 2D Ising models, Fig. 9 plots every hyperparameter trial in the accuracy–cost plane. Both reach a test NLL near 35.5 at comparable kinetic energy. The 1D Ising chain is close to Gaussian, so neither architecture has room to separate. In 2D the sinusoidal-embedding blocks reach a lower NLL (best≈75.1) than the tanh blocks (best≈80.8) at comparable transport cost, mirroring the efficiency advantage seen on the checkerboard.



(a) (b)

Fig. 9. Sinusoidal-embedding versus tanh blocks in neural RG, the two overlap (a) on 1D Ising model, (b) on 2D Ising model.

The per-trial of Fig. 9 are listed in Tables III and IV. In 1D Ising model the best negative log-likelihoods differ by hundredths of a nat. In 2D Ising the best sinusoidal-embedding trial (75.1) beats the best tanh trial (80.8) by about six nats.

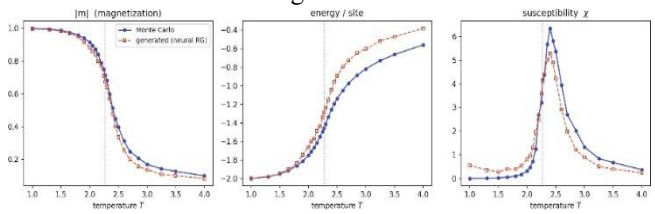
TABLE III. PER-TRIAL RESULTS ON THE 1D ISING RG (L=32).

Trial	1	2	3	4	5
Sinusoidal NLL	35.52	35.52	**35.50**	35.53	35.54
Sinusoidal KE	2.34	1.90	2.26	2.61	1.60
tanh NLL	35.51	35.52	**35.49**	35.50	35.58
Tanh KE	2.19	2.11	2.26	2.26	1.55

TABLE IV. PER-TRIAL RESULTS ON THE 2D ISING RG (16×16).

Trial	1	2	3	4	5	6
Sinusoidal NLL	**75.1**	75.9	75.1	75.3	78.7	82.3
Sinusoidal KE	17.7	16.7	32.2	22.3	5.7	4.1
tanh NLL	81.8	82.2	82.9	**80.8**	82.5	83.2
Tanh KE	15.8	12.6	17.3	18.9	10.7	7.5

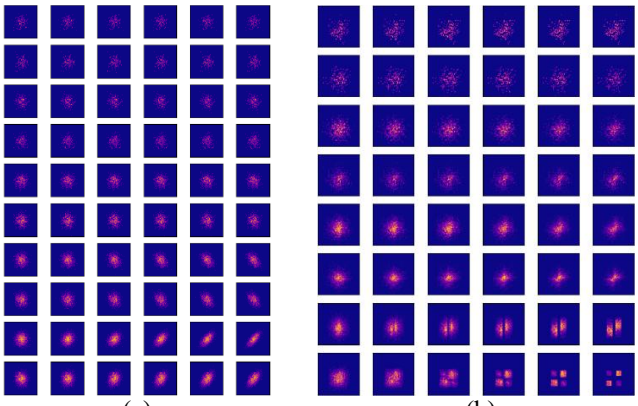
We train a separate model at each of 25 temperatures spanning $T_c \approx 2.269$, generated configurations (red) closely match Monte Carlo (blue) as shown in Fig. 10. Which indicates faithfully reproducing the phase transition occurs just above T_c where fluctuations are strongest.



(a) (b) (c)

Fig. 10. The trained 2D neural RG reproduces the Ising phase transition: (a) magnetization $|m|$, (b) energy per site, (c) susceptibility x.

To visualize what each block learns, Fig. 11 displays for every RG layer of the trained 1D and 2D models, the density of each per-block flow as it transports a Gaussian with distribution over six snapshots per layer. Rows correspond to successive RG layers (each containing a decimator and a disentangler), and columns show six snapshots along flow time. The flows are gentle and low-curvature, consistent with the low transport cost of the sinusoidal embedding. In the 2D case, the coarsest layers also expose the 2×2 block structure of the lattice.



(a) (b)

Fig. 11. Per-layer flows of the trained sinusoidal-embedding neural RG: (a) 1D (L=32), (b) 2D (16×16 Ising).

Our neural RG learns meaningful physics—its coarse-graining hierarchy offers clear interpretability and reproduces the 2D Ising transition (Fig. 10) not just fitting data. However, these findings are specific to our learnable Fourier-feature variant, as tunable frequencies are applied only at the input embedding, not throughout the full depth like SIREN.

V. CONCLUSION

We studied sinusoidal-embedding velocity fields for continuous normalizing flows. Under a fair, early-stopped, three-seed comparison on a deliberately stiff benchmark, a sinusoidal embedding flow matches a tanh baseline in likelihood and sample quality while requiring 25% fewer function evaluations, 14% fewer parameters, and about a third as many training iterations. The frequency scale ω0 controls the accuracy–cost trade-off, and an optimal transport penalty lowers cost further. Used as the block of a neural renormalization group, the same flow ties tanh on the near-Gaussian 1D Ising chain and outperforms it on the richer 2D lattice. Future work includes training the RG variationally from the known Boltzmann distribution rather than from samples.

ACKNOWLEDGMENT

I sincerely thank Professor Peter Lu for proposing the research problem, and for giving insightful suggestions. I'd also thank Oreoluwa Alao for providing me guidance and support. I am grateful to my tutor DJ Liveoak for checking on my project progress, and offering feedback on the presentation and paper. I'd like to thank TAs: Oliver Wang, Celine Zhang, Eegan Ram, Erina Katoh, and Scarlet Gitelson for helping me with coding questions, and providing valuable suggestions for my paper. Finally, I thank Massachusetts Institute of Technology, the Center for Excellence in Education, and PrincetoNow Education Services Inc for providing me the opportunity to research and learn during an unforgettable summer.

REFERENCES

[1] D.J. Rezende and S. Mohamed, "Variational inference with normalizing flows," Intl. Conf. on Mach. Learning, vol. 37, 2015, pp. 1530–1538.

[2] L. Dinh, J. Sohl-Dickstein, and S. Bengio, " Density estimation using Real NVP," Intl. Conf. on Learning Representations, 2017, pp. 1-32.

[3] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural ordinary differential equations," 32nd Conf. on Neural Info. Proc. Sys., 2018, pp. 1-13.

[4] W. Grathwohl, R. T. Q. Chen, J. Bettencourt, I. Sutskever, and D. Duvenaud, "FFJORD: Free-form continuous dynamics for scalable reversible generative models," Intl. Conf. on Learning Representations, 2019, pp. 1-13.

[5] M. Koch-Janusz and Z. Ringel, "Mutual information, neural networks and the renormalization group," Nature Physics, vol. 14, 2018, pp. 578–582.

[6] P.M. Lenggenhager, D.E. Gökmen, Z. Ringel, S.D. Huber, and M. Koch-Janusz, "Optimal renormalization group transformation from information theory," Phys. Rev. X, vol. 10, 2020, 011037.

[7] C. Finlay, J. H. Jacobsen, L. Nurbekyan, and A. Oberman, "How to train your neural ODE: the world of Jacobian and kinetic regularization," Intl. Conf. on Mach. Learning, vol. 119, 2020, pp. 3154-3164.

[8] D. Onken, S. W. Fung, X. Li, and L. Ruthotto, "OT-Flow: Fast and accurate continuous normalizing flows via optimal transport," 35th AAAI Conf. on Art. Intell., vol. 35,2021, pp. 9223-9232.

[9] M. Tancik, P. P. Srinivasan, B. Mildenhall, et al. "Fourier features let networks learn high frequency functions in low dimensional domains," 34th Conf. on Neural Info. Proc. Sys., 2020, pp. 7537-7547.

[10] V. Sitzmann, J. Martel, A. Bergman, D. Lindell, and G. Wetzstein, "Implicit neural representations with periodic activation functions," 34th Conf. on Neural Info. Proc. Sys., 2020, pp. 7462-7473.

[11] J. D. Benamou and Y. Brenier, "A computational fluid mechanics solution to the Monge–Kantorovich mass transfer problem," Numer. Math. vol. 84, 2000, pp. 375–393.

[12] S. H. Li and L. Wang, "Neural network renormalization group," Phys. Rev. Lett., vol. 121, 2018, 260601.

[13] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," Intl. Conf. on Learning Representations, 2015, pp. 1-15.

[14] J. Bradbury, R. Frostig, P. Hawkins, et al. "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: https://github.com/jax-ml/jax.

[15] P. Kidger, On Neural Differential Equations, PhD thesis, University of Oxford, 2021.

[16] A. Gretton, K. M. Borgwardt, M. J. Rasch, et al. "A kernel two-sample test," J. of Mach. Learning Res, vol. 13, 2012, pp. 723–773.